Query cookbook

Ready-made snippets for every lab. Every snippet was tested against the lab files. They show *how* to pivot, not what to find.

Linux command line: jq and grep

Install `jq`:

`sudo apt install jq` on Ubuntu and Debian

`brew install jq` on macOS.

Level 1: read raw events

Run these from `Level1_net_localgroup/`, in this order. Each step adds one idea, and Lab 2 builds on the same steps. The examples use other event types than the tasks ask about, so swap in the field or EventID your task needs. Outputs are left out on purpose.

Step 1. Set up and look at one event. The file has one JSON event per line. Look at a single event first, so you know the field names.

```
cd ~/siem-labs/SIEM_Starter_labs/Level1_net_localgroup
F=$(ls *.json)            # the lab's only .json file
head -1 "$F" | jq .       # the first event, one field per line
head -1 "$F" | jq 'keys'  # just its field names
```

Step 2. Count events. One line is one event, so counting lines counts events.

```
wc -l "$F"
```

Step 3. Print one field from every event. `.Channel` picks a field; `-r` prints it without quotes. Field names with an `@` or a dot need quotes: `."@timestamp"`.

```
jq -r '.Channel' "$F" | head -5
jq -r '."@timestamp"' "$F" | head -5
```

Step 4. Group and count. `sort` puts equal values next to each other, `uniq -c` counts them, `sort -rn` puts the biggest count first. Use the same pattern for any field.

```
jq -r '.Channel' "$F" | sort | uniq -c | sort -rn
```

Step 5. Count two fields together. Put the fields in `[ ]` and join them into one line.

```
jq -r '[.Hostname, .Channel] | join(" | ")' "$F" | sort | uniq -c | sort -rn
```

Step 6. Sort to find the first and last. ISO times such as `2020-09-21T23:18:47Z` sort correctly as text. `sed -n '1p;$p'` prints the first and the last line.

```
jq -r '."@timestamp"' "$F" | sort | sed -n '1p;$p'
```

Step 7. Keep only some events with `select(...)`. `==` means "is exactly"; numbers go without quotes. This example keeps Sysmon 10 (one process opened another).

```
jq -c 'select(.EventID==10)' "$F" | wc -l
jq -r 'select(.EventID==10) | [."@timestamp", .Hostname, .SourceImage, .TargetImage]
| join(" | ")' "$F" | head -3
```

Step 8. Read one whole event of a type. Filter, take the first match, then pretty-print it.

```
jq -c 'select(.EventID==4103)' "$F" | head -1 | jq .
```

Step 9. Search for a word. `grep -i` finds a word anywhere in the line, ignoring case; `-c` counts matching lines. Inside `jq`, `test("word";"i")` does the same for one field.

```
grep -c -i 'powershell' "$F"
jq -r 'select((.Channel//"")|test("powershell";"i")) | .EventID' "$F" | sort | uniq -
c
```

Step 10. Trim long fields. Command lines can be thousands of characters long. `cut -c1-120` shows the first 120.

```
jq -r 'select(.EventID==4103) | .Payload' "$F" | head -3 | cut -c1-120
```

Step 11. Decode Base64 as text, never run it. PowerShell's `-enc` argument is Base64 of UTF-16LE text, so decode it and convert it to normal text. Practise on this harmless sample first.

```
echo 'VwByAGkAdABlAC0ASABvAHMAdAAgACcASABlAGwAbABvACwAIABTAE8AQwAnAA==' | base64 -d

# Write-Host 'Hello, SOC'
```

Lab 2: from a hint to a command

Run these from `Level2B_host_and_network/`. Every Lab 2 hint is one piece of a `jq` command. The steps below build that command one piece at a time, and each step adds one idea. The examples look at ordinary traffic to the domain controller (`192.168.1.4`), not at the attack, so the output gives nothing away.

Step 1. Set up and look at one record. Always look at a single record first, so you know the field names.

```
cd ~/siem-labs/SIEM_Starter_labs/Level2B_host_and_network
H=host/MKT01_windows_events.json   # the workstation's Windows events
Z=zeek                             # the folder with the Zeek logs
head -1 "$Z/conn.log" | jq .       # the first Zeek connection, one field per line
```

Step 2. Pick events with `select(...)`. `select` keeps only the events where the test is true. `==` means "is exactly". Text goes in double quotes, numbers do not. Combine tests with `and` or `or`.

```
# How many Sysmon network connections (EventID 3) went to the domain controller?
jq -c 'select(.EventID==3 and .DestinationIp=="192.168.1.4")' "$H" | wc -l
```

Step 3. Print only the fields you need. `.Image` prints one field. `[.A, .B, .C] | join(" | ")` prints several on one line. `-r` removes the quotes around the text.

```
jq -r 'select(.EventID==3 and .DestinationIp=="192.168.1.4") | [.SourcePort,
.DestinationPort, .Image] | join(" | ")' "$H" | head -3
# 63829 | 53 | C:\Windows\System32\svchost.exe
```

Step 4. Count instead of reading. `sort | uniq -c` counts how often each value appears; `sort -rn` puts the biggest count first.

```
jq -r 'select(.EventID==3 and .DestinationIp=="192.168.1.4") | .DestinationPort' "$H"
| sort | uniq -c | sort -rn
#  67 53     (DNS)
#  13 88     (Kerberos) ...
```

Step 5. "Contains" instead of "is exactly". `test("word";"i")` is true when the field contains the word, ignoring upper and lower case. `(.Image//"")` means "use empty text if the field is missing", so `jq` does not stop with an error.

```
jq -r 'select(.EventID==1 and ((.Image//"")|test("powershell";"i"))) | .CommandLine'
"$H" | head -2
```

Step 6. Zeek: quote names with a dot, convert the time. Zeek field names such as `id.resp_h` contain a dot, which `jq` would read as "a field inside a field", so put the name in quotes: `."id.resp_h"`. `ts` is seconds since 1970; `todate` turns it into a readable UTC time. `{name: value}` prints a small labelled record.

```
jq -c 'select(."id.resp_h"=="192.168.1.4") | {time: (.ts|todate), from_port:
."id.orig_p", to_port: ."id.resp_p"}' "$Z/conn.log" | head -3
# {"time":"2023-08-16T04:53:42Z","from_port":52605,"to_port":53}
```

Step 7. Join Zeek to the host on the source port. The same connection is in both logs: Zeek's `id.orig_p` is the host's `SourcePort`. Zeek stores the port as a number and Sysmon as text, so write it without quotes for Zeek and with quotes for Sysmon.

```
jq -c 'select(."id.orig_p"==61988) | {time: (.ts|todate), server: ."id.resp_h", port:
."id.resp_p"}' "$Z/conn.log"
jq -c 'select(.EventID==3 and .SourcePort=="61988") | {time: ."@timestamp", Image,
User}' "$H"
# Same connection, two times about 19 hours apart: that is why you join on the port,
not the time.
```

More Lab 2 patterns

```
# Every host event that carries one source port: Sysmon 3, and Security 5156/5158
from the Windows firewall
jq -c 'select(((.SourcePort//"")|tostring)=="61988") | {time: ."@timestamp", Channel,
EventID, Image, Application}' "$H"

# Show a Zeek time on the host clock (Zeek is about 19 hours = 68,400 seconds ahead)
jq -c 'select(."id.orig_p"==61988) | {zeek_time: (.ts|todate), host_time: (.ts -
68400 | todate)}' "$Z/conn.log"

# All connections to one server, in time order (change S to the server you are
looking at)
S=192.168.1.4
jq -c --arg s "$S" 'select(."id.resp_h"==$s) | {time: (.ts|todate), from_port:
."id.orig_p", to_port: ."id.resp_p", bytes_back: (.resp_bytes//0)}' "$Z/conn.log" |
sort

# Which event types are most common in the host log? Count first, then read single
events
jq -r '.EventID' "$H" | sort -n | uniq -c | sort -rn | head
```

Lab 2 stretch: one file, many Zeek logs

Run these from `Level2A_zeek_apt29_day2/`. The examples use the domain controller's DNS traffic.

```
F=combined_zeek.log

# which Zeek logs are in the file
jq -r '."@stream"' "$F" | sort | uniq -c | sort -rn | head -4

# one record, to see its field names
jq -c 'select(."@stream"=="dns")' "$F" | head -1

jq -r 'select(."@stream"=="conn" and .id_orig_h=="10.0.1.4") |
"\(.id_resp_h):\(.id_resp_p)"' "$F" | sort | uniq -c | sort -rn | head -3

jq -r 'select(."@stream"=="conn" and .id_resp_h=="10.0.0.4" and .id_resp_p==53) |
.ts' "$F" | sort -n
```

Level 3: first look

Run these from `Level3_apt29_day1/`. Start with the overview (count, time range, hosts, channels), then pivot.

```
F=apt29_evals_day1_manual_2020-05-01225525.json
SYSMON='Microsoft-Windows-Sysmon/Operational'

# Count, time range, events per host
wc -l < "$F"
jq -r '.EventTime' "$F" | sort | sed -n '1p;$p'
jq -r '.Hostname' "$F" | sort | uniq -c | sort -rn

# Events per Channel + EventID
jq -r '[.Channel, (.EventID|tostring)] | join("\t")' "$F" | sort | uniq -c | sort -rn
| head -20

# Process creation on one host, in time order
jq -r --arg s "$SYSMON" 'select(.Channel==$s and .EventID==1 and
(.Hostname|startswith("SCRANTON")))
  | [.EventTime, .User, .IntegrityLevel, ((.ParentImage//"")|split("\\")|last),
.CommandLine]
  | join("\t")' "$F" | sort | less -S

# Unique PowerShell script blocks (first 120 chars), oldest first
jq -r 'select(.EventID==4104 and .ScriptBlockText)
  | [.EventTime, .Hostname, (.ScriptBlockText|.[0:120]|gsub("[\r\n\t]+";" "))]
  | join("\t")' "$F" | sort -t$'\t' -u -k3,3 | sort | less -S

# Network connections grouped by host, image, destination
jq -r --arg s "$SYSMON" 'select(.Channel==$s and .EventID==3)
  | [.Hostname, .Image, .DestinationIp, (.DestinationPort|tostring)] | join("\t")'
"$F" \
  | sort | uniq -c | sort -rn | head -30

# Handles opened to lsass
jq -r 'select(.EventID==10 and ((.TargetImage//"")|test("lsass";"i")))
  | [.Hostname, .SourceImage, .GrantedAccess] | join("\t")' "$F" | sort | uniq -c |
sort -rn

# Logons from the Security channel (it appears as "Security" and "security")
jq -r 'select((.Channel|ascii_downcase)=="security" and .EventID==4624)
  | [.Hostname, (.LogonType|tostring), .TargetUserName, .IpAddress] | join("\t")'
"$F" \
  | sort | uniq -c | sort -rn

# Everything on one host in a time window
jq -r 'select(.EventTime>="2020-05-01 22:58:00" and .EventTime<="2020-05-01 23:00:00"
  and (.Hostname|startswith("SCRANTON"))) | [.Channel, (.EventID|tostring)] |
join("\t")' "$F" \
  | sort | uniq -c | sort -rn

# Fast keyword hunt: grep narrows the lines, jq parses only those
grep -F '"EventID":4104' "$F" \
  | jq -r 'select((.ScriptBlockText//"")|test("compress";"i")) | [.EventTime,
```

```
.Hostname] | join("\t")'

# Export to CSV for a spreadsheet or your report
jq -r --arg s "$SYSMON" 'select(.Channel==$s and .EventID==1)
  | [.EventTime, .Hostname, .User, .ParentImage, .Image, .CommandLine] | @csv' "$F" >
processes.csv

# Pretty-print one full event to see all its fields
sed -n '1000p' "$F" | jq .
```

Tips:

- Use `join("\t")`, not `@tsv`. `@tsv` doubles every backslash in Windows paths.
- Guard optional fields with `//""` (for example `(.ParentImage//"")`); otherwise `jq` stops on events where the field is missing.
- To decode a Base64 `-e` argument as text without running it: `echo '<b64>' | base64 -d | iconv -f UTF-16LE -t UTF-8`.

Level 3: group, follow, decode

Run these from `Level3_apt29_day1/`. The examples use routine activity (the domain controller's own connections and a WMI process on UTICA) and a harmless sample string.

```
F=apt29_evals_day1_manual_2020-05-01225525.json

# Group: filter with jq -c, then group only the matches with jq -s (first seen and
count per destination)
jq -c 'select(.EventID==3 and (.Hostname|startswith("NEWYORK")) and
.Initiated=="true")' "$F" \
  | jq -rs 'group_by([.DestinationIp, .DestinationPort])[] | [(map(.EventTime)|min),
"\(.[0].DestinationIp):\(.[0].DestinationPort)", (length|tostring)] | join(" | ")' |
sort | head

# Follow one process: every event that carries its ProcessGuid (as itself, as parent,
as source or target)
G=$(jq -r 'select(.EventID==1 and (.Hostname|startswith("UTICA"))) | .ProcessGuid'
"$F" | head -1)
jq -r --arg g "$G" 'select(.ProcessGuid==$g or .ParentProcessGuid==$g or
.SourceProcessGUID==$g or .TargetProcessGUID==$g)
  | [.EventTime, (.EventID|tostring), (.Image // .SourceImage // "")] | join(" | ")'
"$F" | head

# Decode a second layer as text, never run it: Base64 of gzip
echo 'H4sIAAAAAAAAAAwsvyixJ1fUvLSkoLVFQykjNyclXSCvKz1XISaxMLVIwUgIAhP2aEiEAAAA=' |
base64 -d | gunzip; echo

# Look at raw bytes when a name prints oddly
jq -r 'select(.EventID==1 and (.Hostname|startswith("UTICA"))) | .Image' "$F" | head
-1 | od -An -tx1c | head -4
```

Pivot cheat sheet

Question	Channel or log	EventID	Key fields
What ran, who ran it, from which parent?	Sysmon	1	`Image`, `CommandLine`, `ParentImage`, `User`, `IntegrityLevel`, `Hashes`, `Company`, `OriginalFileName`
Where did a process connect?	Sysmon	3	`Image`, `SourcePort`, `DestinationIp`, `DestinationPort`
Which DLLs did a process load?	Sysmon	7	`Image`, `ImageLoaded`
Did one process inject code into another?	Sysmon	8	`SourceImage`, `TargetImage`, `TargetProcessId`
Who opened a handle to another process (for example lsass)?	Sysmon	10	`SourceImage`, `TargetImage`, `GrantedAccess`, `CallTrace`
Which files were created or deleted?	Sysmon	11, 23	`Image`, `TargetFilename`
Which registry keys were created or set?	Sysmon	12, 13	`Image`, `TargetObject`, `Details`
Where did a downloaded file come from?	Sysmon	15	`TargetFilename`, `Contents` (`HostUrl`, `ZoneId`)
What PowerShell code actually ran?	Microsoft-Windows-PowerShell/Operational	4104	`ScriptBlockText`, `Path`
Which cmdlets ran, with which parameters?	Microsoft-Windows-PowerShell/Operational; Windows PowerShell	4103; 800	`Payload`
Who logged on, how and from where?	Security	4624, 4648	`TargetUserName`, `LogonType`, `IpAddress`, `TargetServerName`
Who listed the members of a local group?	Security	4799	`TargetUserName`, `SubjectUserName`, `CallerProcessName`

Question	Channel or log	EventID	Key fields
Which program did the Windows firewall let connect?	Security	5156	`Application`, `SourcePort`, `DestAddress`, `DestPort`
Was a service installed?	System; Security	7045; 4697	`ServiceName`, `ImagePath`, `ServiceFileName`
Which network shares and files were accessed remotely?	Security	5140, 5145	`ShareName`, `RelativeTargetName`, `IpAddress`
Which connections did the network see?	Zeek `conn.log`		`id.orig_h`, `id.orig_p`, `id.resp_h`, `id.resp_p`, `orig_bytes`, `resp_bytes`, `history`
What came over plain HTTP?	Zeek `http.log`, `files.log`, `pe.log`		`status_code`, `resp_mime_types`, `response_body_len`
Which server and certificate did a TLS session use?	Zeek `ssl.log`, `x509.log`		`server_name`; `certificate.subject`, `certificate.issuer`

A useful habit: once you find one suspicious event, take its `ProcessGuid` and search for every event with the same value. Do the same with its `ParentProcessGuid` and the parent's parent. This rebuilds the full chain.

Reading the values

Field	Value	Means
`GrantedAccess` (Sysmon 10)	contains `0x10` (for example `0x1010`, `0x1410`)	Can read the target's memory
`GrantedAccess`	`0x1000`, `0x1400`	Can only query information about the target
`GrantedAccess`	`0x1FFFFF`	Full access
`IntegrityLevel` (Sysmon 1)	`Medium`, `High`, `System`	Normal user, run as administrator, SYSTEM
`ZoneId` (Sysmon 15 `Contents`)	`3`	Downloaded from the Internet (`0` this computer, `1` intranet, `2` trusted sites)
`LogonType` (Security 4624)	`2`, `3`, `10`	At the keyboard, over the network, Remote Desktop

Field	Value	Means
`history` (Zeek `conn`)	uppercase, lowercase letters	Packets from the client, packets from the server; a leading `^` means Zeek flipped the direction
`orig_bytes`, `resp_bytes` (Zeek `conn`)	numbers	Bytes the client sent, bytes the server sent
`certificate.subject`, `certificate.issuer` (Zeek `x509`)	the same	Self-signed: no certificate authority vouches for the server